

프로젝트 보고서



김명준

목차

1. 프로젝트 개요	3
2. 개발 일정	3
3. 세부 구현 내용	4
4. 문제점 및 보완 사항	5
5. 폴더 구조 및 UI	6

1. 프로젝트 개요

1.1 프로젝트 목적

TCP/IP 소켓 통신을 활용하여 라즈베리파이 4에 연결된 하드웨어 모듈을 원격으로 제어 및 모니터링하는 시스템 구축

1.2 클라이언트 환경

SIGINT 외 모든 시그널을 차단해 안정성을 높인 리눅스 기반 텍스트 클라이언트 및 실시간 연동이 가능한 웹 클라이언트 동시 지원

1.3 소프트웨어 설계

하드웨어 제어 로직을 독립적인 동적 라이브러리(.so)로 분리하여 유지보수성과 확장성 극대화 및 상시 동작을 위한 백그라운드 데몬 프로세스 적용

1.4 주요 제어 장치

- LED: On/Off 및 3단계 밝기 조절 (최저/중간/최대)
- 부저: 음악 On/Off
- 조도 센서: 실시간 밝기 모니터링 및 LED 자동 제어 연동
- 7 세그먼트: 타이머 카운트다운 출력 및 0 도달 시 부저 연동

2. 개발 일정

개발 기간: 2026.06.02. ~ 2026.06.05. (총 4일)

일차	단계	핵심 수행 내용
1일차	요구사항 정의 및 환경 구축	통신 프로토콜 정의 Makefile 빌드 자동화 스크립트 작성 하드웨어 모듈 단위 테스트
2일차	서버 데몬 및 라이브러리 개발	하드웨어 동적 라이브러리(.so) 추출 서버 데몬 프로세스 전환 epoll 기반 비동기 이벤트 루프 구축
3일차	클라이언트 구현 및 웹 연동	리눅스 클라이언트 시그널 블로킹 클라이언트 비동기 I/O 구현 HTTP 파싱 및 SSE 적용
4일차	통합 테스트 및 디버깅	엣지 케이스 테스트 산출물 정리 및 최종 문서화

3. 세부 구현 내용

3.1 비동기 이벤트 루프 아키텍처

- 서버 (epoll 기반 싱글 스레드 아키텍처)

- 도입 배경: 다중 클라이언트 접속 시 클라이언트마다 스레드를 생성할 경우, 잦은 문맥 교환으로 인해 서버에 막대한 오버헤드가 발생
- 해결 방안: 기존의 멀티 스레드 기반 연결 방식을 과감히 폐기하고, 리눅스의 고성능 I/O 다중화 기법인 epoll을 적용
- 기대 효과: 싱글 스레드 기반의 비동기 이벤트 루프로 동작하여, 최소한의 시스템 자원만으로도 다수의 네트워크 연결을 지연 없이 고속으로 처리

- 리눅스 클라이언트 (select 기반 비동기 I/O)

- 도입 배경: 사용자 키보드 입력을 대기하는 동안 서버의 응답을 수신하지 못하거나, 그 반대로 발생하는 블로킹 현상을 방지해야 함
- 해결 방안: select 시스템 콜을 활용하여 표준 입력과 소켓 수신 이벤트를 동시에 감시하는 구조 적용
- 기대 효과: 사용자의 명령어 입력과 서버 측의 상태 데이터 수신을 상호 간섭이나 프로젝트 멈춤 현상 없이 실시간으로 처리

3.2 SSE 기반 실시간 통신

- 도입 배경 및 웹소켓의 한계

- 도입 배경: 하드웨어 장치의 상태 변화를 웹 브라우저 화면에 즉각적으로 반영하기 위해서는 서버가 클라이언트 측으로 데이터를 밀어주는 실시간 지속 연결이 필수적임
- 웹소켓 배제 사유: 실시간 양방향 통신인 웹소켓이 대표적인 해결책이나, 순수 C언어로 웹소켓의 복잡한 핸드셰이크 및 데이터 프레임밍을 직접 구현하는 것은 불필요한 시스템 오버헤드와 복잡도를 유발한다고 판단함

- 해결 방안 (SSE 단방향 아키텍처 채택)

- 해결 방안: 웹소켓의 대안으로 기존 HTTP 인프라 위에서 가볍게 동작하는 브라우저 네이티브 기술인 SSE를 채택. 장치 제어는 비동기 HTTP POST로, 상태 모니터링은 SSE를 통한 단방향 수신으로 역할을 완전히 분리함
- 기대 효과: 서버 코어에 복잡한 프로토콜 구현을 추가할 필요 없이, 간단한 HTTP 헤더(Content-Type: text/event-stream)만으로 다수의 웹 클라이언트에 장치 상태(JSON)를 지연 없이 가볍고 빠르게 브로드캐스트하는 효율적인 실시간 관제 환경을 완성

3.3 상태 중앙 집중화 및 스레드 동기화

- 상태 중앙 집중화 (device_state 전역 구조체)

- 도입 배경: 하드웨어 모듈마다 상태를 개별적으로 관리할 경우 제어 코드가 파편화되며, 다수의 웹 클라이언트에게 전체 상태를 전송할 때 데이터를 일일이 취합해야하는 병목이 발생함

- **해결 방안:** 시스템 내 모든 하드웨어의 상태와 제어 플래그를 통합한 단일 전역 구조체 `device_state`를 설계하며, 이를 유일한 상태 저장소로 활용
- **기대 효과:** 스레드 생성 및 하드웨어 제어 함수의 인터페이스가 통일되어 유지보수성이 극대화되며, 웹 서버는 해당 구조체 하나만 JSON으로 직렬화하여 전송하면 되므로 브로드캐스트 아키텍처가 매우 단순해지고 응답 속도가 향상됨
- **플래그 패턴 기반 스레드 동기화 (락 점유 최소화)**
 - **도입 배경:** 비동기 네트워크 루프와 다수의 하드웨어 제어 워커 스레드가 단일 전역 구조체에 동시 접근함에 따라 발생하는 교착 상태 위험을 방지해야 함
 - **해결 방안:** mutex를 도입하여 스레드를 동기화하되, 임계 구역 내부에서는 상태 값만 빠르게 갱신하고 빠져나오는 플래그 패턴을 적용
 - 락이 완전히 해제된 이후에 브로드캐스트를 수행하도록 실행 시점을 완벽히 분리함
 - **기대 효과:** 락 점유 시간을 최소화하여 시스템 전체의 I/O 블로킹 현상을 방지하고, 멀티 스레드 환경에서의 데이터 정합성과 서버의 구동 안정성을 동시에 확보

3.4 이중 연결 리스트 기반 고속 세션 관리

- **도입 배경 및 기존 자료구조의 한계**
 - **도입 배경:** epoll 기반의 싱글 스레드 서버에서는 다수의 클라이언트 접속 및 종료 이벤트가 빈번하게 발생하며, 특정 클라이언트의 연결 종료 시 세션을 즉각적으로 해제하고 자원을 반환해야 함
 - **기존 한계:** 배열이나 단일 연결 리스트를 사용할 경우, 삭제할 세션을 탐색하고 메모리를 재배열하는 데 $O(N)$ 의 시간 복잡도가 발생하여 고성능 비동기 이벤트 루프에 치명적인 병목 현상을 유발할 위험이 존재함
- **해결 방안 (이중 연결 리스트 적용)**
 - **해결 방안:** 클라이언트 세션 관리에 이중 연결 리스트 자료구조를 도입. 특정 클라이언트의 연결 종료 요청이 감지되면, 해당 노드의 이전 및 다음 노드 포인터만 재연결하여 즉각적으로 연결을 끊어내는 로직을 구현함
 - **기대 효과:** 세션 삭제 및 자원 반환 과정을 $O(1)$ 의 시간 복잡도로 극적 최적화하여 이벤트 루프의 지연을 원천 차단함. 이를 통해 메모리 누수 및 좀비 소켓 발생을 완벽하게 방지하고 서버의 장기 구동 안전성을 확보함

4. 문제점 및 보완 사항

4.1 하드웨어 제어 (7세그먼트 핀 매핑 및 신호 역전 현상)

- **문제 현상 및 원인:** 초기 회로 설계 시 7세그먼트 모듈을 공통 음극 방식으로 오인하여 공통 핀을 GND에 연결하였으나, 디스플레이가 전혀 출력되지 않는 문제가 발생함. 회로 검토 결과, 실제 사용된 소자가 공통 양극 방식임이 확인됨
- **해결 방안 및 결과:** 하드웨어 단에서는 공통 핀의 결선을 GND에서 5V 전원으로 재배치함
소프트웨어 단에서는 변경된 하드웨어 특성에 맞추어 GPIO 출력 제어 로직을 수정, 기존의 High/Low 제어 신호를 완벽히 반전시킴으로써 정상적인 타이머 출력을 구현함

4.2 네트워크 통신 (웹 클라이언트 실시간 동기화 한계 극복)

- **문제 현상 및 원인:** 초기 웹 대시보드 연동 시, 일반적인 HTTP 프로토콜의 단방향 특성으로 인해 잦은 문제에 부딪힘. 다른 클라이언트나 연동된 센서에 의해 장치 상태가 변하더라도, 사용자가 직접 브라우저를 새로고침하기 전까지는 화면이 갱신되지 않는 치명적인 한계가 있었음.
- **해결 방안 및 결과:** 실시간 상태 동기화를 위해 서버와 브라우저 간 지속 연결을 유지하는 SSE 기술을 서버 코어 루프에 직접 구현함. 장치 상태에 변화가 발생하면 서버가 주도적으로 연결된 모든 클라이언트에게 JSON 형태로 데이터를 즉각 브로드캐스트하도록 통신 아키텍처를 전면 개편하여, 지연 없는 실시간 관제 UI를 완성함.

5. 폴더 구조 및 UI

- 폴더 구조

├─ Makefile	# 전체 프로젝트(client, device, server) 통합 빌드 자동화 스크립트
├─ README.md	# 프로젝트 개요, 개발 일정, 트러블슈팅 등 통합 문서
├─ bin	# 빌드된 최종 실행 파일(client, server)과 라이브러리(librpi.so)가 모이는 폴더
├─┬─ index.html	# 웹 페이지
├─ client	# 텍스트 기반 리눅스 TCP 클라이언트 프로그램
├─┬─ Makefile	# 클라이언트 전용 빌드 스크립트
├─┬─ include	
├─┬─┬─ menu.h	# 사용자 인터페이스(CLI 메뉴) 관련 헤더
├─┬─┬─ setup.h	# 클라이언트 설정 관련 헤더
├─┬─ src	
├─┬─┬─ main.c	# 클라이언트 프로그램 진입점 (main 함수)
├─┬─┬─ menu.c	# 콘솔 UI 출력 및 사용자 입력(target/action) 파싱 로직
├─┬─┬─ setup.c	# 서버 연결 및 시그널 설정
├─ device	# 하드웨어 제어 라이브러리 (서버와 독립적으로 분리된 모듈)
├─┬─ Makefile	# librpi.so 동적 공유 라이브러리 빌드 스크립트
├─┬─ buzzer.c	# 부저 제어 및 상태 갱신 로직
├─┬─ cds.c	# 조도 센서(I2C) 제어
├─┬─ led.c	# LED 제어 (소프트웨어 PWM 활용)
├─┬─┬─ segment.c	# 7-세그먼트 타이머 출력 및 0 도달 시 부저 연동 로직
├─ scp.sh	# 빌드 완료 후 라즈베리파이 원격 서버로 파일 자동 배포 스크립트
├─ server	# epoll 기반 고성능 비동기 TCP/HTTP 제어 서버
├─┬─ Makefile	# 서버 전용 빌드 스크립트
├─┬─ include	
├─┬─┬─ core.h	# 서버 네트워크 뼈대(소켓, epoll 이벤트 루프, 데몬화) 헤더
├─┬─┬─ handler.h	# 클라이언트 명령 파싱 및 하드웨어 디스패처 헤더
├─┬─┬─ list.h	# 브로드캐스트용 클라이언트 세션 관리를 위한 이중 연결 리스트 헤더
├─┬─┬─ state.h	# 하드웨어 통합 상태 구조체 설계도 및 extern 선언부
├─┬─┬─ thread.h	# 동적 라이브러리 로드 및 워커 스레드 관리 헤더
├─┬─ src	
├─┬─┬─ core.c	# 백그라운드 데몬 전환, epoll 비동기 이벤트 루프 및 TCP/HTTP 세션 통합 처리부
├─┬─┬─ handler.c	# 수신 문자열 분기 처리 및 하드웨어 워커 스레드 생성(상태 선반영) 로직
├─┬─┬─ list.c	# 세션(Session) 삽입/삭제를 O(1)로 처리하기 위한 이중 연결 리스트 구현부
├─┬─┬─ server.c	# 서버 프로그램 진입점 (main 함수, 소켓 초기화 및 루프 실행)
├─┬─┬─ state.c	# 전역 상태(g_state) 메모리 실제 할당 및 초기값 세팅
├─┬─┬─ thread.c	# dlopen을 이용한 librpi.so 동적 로드 및 하드웨어 제어 함수 비동기 실행 로직

- UI

- CLI 환경

```
=====
1. LED
2. BUZZER
3. CDS
4. SEGMENT
5. EXIT
=====
Input > █
```

- 웹 사이트

